Deli2

https://gemini.google.com/app/c0a1aed066996ff4

User prompt: Jag vill ha en roadmap för att kunna lösa problemen i filen, kan du hjälpa mig med det? Jag kan skriva koden själv så kan jag skicka koden till dig för att dubbel checka med dig.

Response: Här är en steg-för-steg-översikt och implementeringsplan för uppgiften. Den är uppdelad så att du kan bygga och testa funktionerna en i taget och skicka koden hit för granskning efter hand.

Fas 0: Förberedelser och miljö

1. **Skapa virtuell miljö och installera paket:**
    - Skapa en ny virtuell miljö (t.ex. `python -m venv .venv`).
    - Installera nödvändiga bibliotek: `pandas`, `numpy`, `jupyter`.
    - Exportera miljön direkt till en fil: `pip freeze > requirements.txt`.
2. **Generera testdata:**
    - Kör `python generate_data_for_d2.py` för att bygga mappstrukturen under `data/` samt `data/store_config.json`.

Fas 1: Task 1 – Filläsning och regex

1. **Robust filläsare (`read_log_file`):**
    - Ta in en `pathlib.Path`.
    - Försök läsa filen med UTF-8 (`try ... except UnicodeDecodeError`). Om det kraschar, läs med ISO-8859-1.
    - Returnera en tupel: `(innehåll, använd_encoding)`.
2. **Regex-parser (`parse_log_text`):**
    - Definiera ett reguljärt uttryck som matchar mallen: `\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] \[([A-Za-z0-9_]+)\] item=(\w+) price=([\d\.]+) (\w+) \(([A-Z]+)\)`
    - Loopa igenom rader eller använd `re.finditer`. Bruslinjer ignoreras automatiskt då de inte matchar mönstret.
    - Konvertera fälten till rätt typer (`price` till `float`, övriga till `str`) och returnera en lista med dictionaries.

Fas 2: Task 2 – Rekursiva hjälpfunktioner

1. **Rekursiv filsökning (`find_log_files`):**
    - Ta in en sökväg och konvertera till `pathlib.Path`.
    - Loopa igenom katalogen med `.iterdir()`.
    - Om objektet är en mapp: anropa funktionen rekursivt.
    - Om objektet är en fil och slutar på `.log`: spara i en lista.
    - *Regel:* Använd **inte** `os.walk`, `Path.glob` eller `Path.rglob`.

2. **Rekursiv sökning i JSON-struktur (`get_active_registers`):**
    - Läs in `store_config.json` med `json.load()`.
    - Skriv en funktion som tar emot godtycklig datastruktur (`dict`, `list`, eller skalära värden):
        - Om det är ett `dict`: kontrollera om det innehåller `"id"` och `"active": True`. Om ja, spara id:t. Loopa sedan över alla dess värden rekursivt.
        - Om det är en `list`: loopa över alla element och anropa funktionen rekursivt.
    - Returnera ett `set[str]` med alla aktiva register-ID:n.

Fas 3: Task 3 – Rengöring och beräkning med NumPy

1. **Identifiera och ersätta outliers (`clean_price_outliers`):**
    - Konvertera listan eller serien till en 1D NumPy float-array.
    - Beräkna medelvärde (μ) och standardavvikelse (σ).
    - Identifiera outliers med villkoret $|x - \mu| > 3\sigma$.
    - Beräkna medianen av alla **giltiga** priser (dvs. exklusive outliers) och ersätt outlier-värdena med denna median.
    - Returnera den rengjorda arrayen.
2. **Beräkna refund ratio (`compute_refund_ratio`):**
    - Implementera formeln elementvis:

$$\text{Refund Ratio} = \frac{\text{refund_count}}{\text{sales_count} + 10^{-6}}$$

    - Säkerställ att den tar emot och returnerar NumPy-arrays.

Fas 4: Task 4 – Pandas-analys

1. **Skapa DataFrame (`create_transaction_dataframe`):**
    - Läs in dictionary-listan till en `pd.DataFrame`.
    - Konvertera kolumnen `timestamp` till `pd.to_datetime`.
    - Sätt `timestamp` som index och sortera i kronologisk ordning.
2. **Aggregera per timme (`resample_hourly_revenue`):**
    - Gruppera efter `item` och kör en resample på 1-timmarsintervall (`'1h'`) på tidsstämpel-indexet.
    - Summera `price` för att beräkna total intäkt.
3. **Skapa pivot-tabell (`create_status_pivot`):**
    - Bygg en pivot-tabell (`df.pivot_table`) med `item` som rader (index), `status` som kolumner (`SUCCESS`, `REFUND`, `CANCELLED`) och summan av `price` som värden.
4. **Beräkna refund ratio per vara (`calculate_item_refund_ratios`):**

- Räkna antalet rader med status `SUCCESS` per vara respektive antalet med status `REFUND` per vara.
- Skicka arrayerna till din funktion från Task 3 (`compute_refund_ratio`) och sammanställ resultatet i en DataFrame.

Fas 5: Task 5 – Hela arbetsflödet och export

1. **Koppla ihop flödet i en huvudpipeline:**
   - Hitta alla `.log`-filer rekursivt och hämta aktiva register från JSON.
   - Läs och parsa alla loggfiler till en samlad transaktionslista.
   - *(Valfritt/kontrollera spec):* Filtrera eventuellt bort transaktioner från inaktiva kassaapparater om uppgiften kräver att endast aktiva register behålls.
   - Bygg Pandas DataFrame.
   - Ersätt kolumnen `price` med resultatet från din outlier-funktion i NumPy.
   - Valutakorrigering: Multiplicera `price` med 10.0 där `currency == 'NOK'`.
2. **Exportera filer:**
   - `cleaned_transactions.csv`
   - `hourly_revenue.json`
   - `status_pivot_table.csv`
   - `item_refund_ratios.csv`

Fas 6: Slutförande och inlämning

- Kör igenom hela Jupyter-notebooken från topp till botten utan fel.
- Exportera notebooken till PDF.
- Skapa `git_log.txt` via terminalkommandot: `git log --graph --pretty=format:"%h - %an, %ar : %s" > git_log.txt`
- Förbered text om användning av AI-verktyg.
- Spela in den korta presentationsvideon (ca 5 min).

Vilken del (till exempel Task 1 med filinläsning och regex) vill du börja skriva kod för först?

User prompt: Fas 0 är faridg, vi kan gå till fas 1

Response: Perfekt! Då kör vi igång med **Task 1: File ingestion and regular expressions**.

I den här delen ska du skriva två funktioner:

1. Robust filläsare (`read_log_file`)

- **Indata:** Ett sökvägsobjekt från modulen `pathlib.Path`.
- **Krav:**

    - Funktionen ska automatiskt hantera filkodning utan att du hårdkodar vilken fil som har vilken kodning.
    - De två formaten som ska stödjas är `UTF-8` och `ISO-8859-1` (Latin-1).
    - *Tips:* Använd en `try-except`-struktur där du först provar att läsa filen med UTF-8, och fångar `UnicodeDecodeError` för att sedan läsa med ISO-8859-1.
- **Utdata:** En tupel med två strängar: `(text_innehall, vald_encoding)`.

2. Regex-parser (`parse_log_text`)

- **Indata:** Textinnehållet (som en sträng) från en loggfil.
- **Format på giltig rad:** `[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)`
- **Rader att ignorera (brus):** `--- PRINTER PAPER LOW WARNING ---` eller `=== NETWORK SYNC OK ===`
- **Fält att extrahera med regex grupper (`re`):**
    1. `timestamp` (sträng: `YYYY-MM-DD HH:MM:SS`)
    2. `register_id` (sträng, t.ex. `POS_01`)
    3. `item` (sträng, t.ex. `Latte`)
    4. `price` (flyttal / `float`, t.ex. `4.50`)
    5. `currency` (sträng, t.ex. `EUR`, `NOK`, `USD`)
    6. `status` (sträng: `SUCCESS`, `REFUND` eller `CANCELLED`)
- **Utdata:** En lista med dictionaries (`list[dict]`), där varje dictionary motsvarar en godkänd rad och har de 6 fälten som nycklar.

Skriv ditt utkast för dessa två funktioner och klistra in koden här så granskar vi den tillsammans! Vill du ha ett förslag på hur regex-mönstret kan byggas upp innan du börjar koda, eller vill du prova att skriva båda funktionerna direkt?

User prompt: import re import json from pathlib import Path import numpy as np import pandas as pd

Response: Importen ser helt rätt ut för allt som behövs i uppgiften.

Du kan nu börja med den första funktionen för Task 1, filläsaren:

- Ta in en sökväg och säkerställ att den konverteras till ett `Path`-objekt om den inte redan är det.
- Försök läsa med `utf-8`.
- Vid fel (`UnicodeDecodeError`), läs med `iso-8859-1`.
- Returnera en tupel `(content, encoding)`.

Klistra in din funktion när du har skrivit den!

Vill du skriva funktionen för filläsaren direkt, eller har du några funderingar kring hur `try/except` med `read_text` bäst struktureras?

User prompt: #Task 1.1 def read_log_file(path) -> tuple[str, str]: #read a text file, trying UTF-8 first and falling back to latin-1 path = Path(path) try: text = path.read_text(encoding="utf-8") return text, "utf-8" except UnicodeDecoderError: text = path.read_text(encoding="iso-8859-1") return text, "iso-8859-1"

Response: Koden är nästan helt perfekt och strukturen är helt rätt tänkt! Det finns bara ett litet stavfel i exception-namnet:

Det heter `UnicodeDecodeError` (med **e**, inte *er* på slutet). Om du fångar `UnicodeDecoderError` kastar Python ett `NameError` eftersom den klassen inte existerar.

Den korrigerade versionen ser ut så här:

```
# Task 1.1
def read_log_file(path) -> tuple[str, str]:
    # read a text file, trying UTF-8 first and falling back to latin-1
    path = Path(path)
    try:
        text = path.read_text(encoding="utf-8")
        return text, "utf-8"
    except UnicodeDecodeError:
        text = path.read_text(encoding="iso-8859-1")
        return text, "iso-8859-1"
```

Denna uppfyller alla krav för del 1.1:

- Konverterar till `Path`.
- Försöker läsa `utf-8` utan att hårdkoda.
- Faller tillbaka till `iso-8859-1` vid avkodningsfel.
- Returnerar en tupel med texten och namnet på den encoding som användes.

Nu kan du gå vidare till **Task 1.2: Regex-parsern**.

Vill du skriva regex-funktionen som parsar texten till en lista av dictionaries nu och skicka över koden?

User prompt: text, enc= read_log_file("data/2026/library_cafe/POS_01/register_sales.log") print("Encoding:", enc) print(text[:400])

Response: Att testa direkt mot en genererad fil är ett utmärkt sätt att verifiera funktionen på.

När du kör den källkoden i din notebook kommer du att få ut:

- Vilken encoding filen lästes med (`utf-8` eller `iso-8859-1`).
- De första 400 tecknen av loggfilen så att du kan inspektera hur datan och brusraderna faktiskt ser ut.

Om utskriften visar texten utan fel är du redo för **Task 1.2: `parse_log_text`**. Kom ihåg kraven för den funktionen:

- Bygg ett reguljärt uttryck som matchar giltiga rader enligt mönstret: `[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)`.
- Fånga 6 fält: `timestamp`, `register_id`, `item`, `price`, `currency`, `status`.

- Konvertera `price` till `float`.
- Returnera en lista med dictionaries.

Fungerade utskriften som förväntat, och vill du skriva regex-parsern för Task 1.2 härnäst?

User prompt: LINE_PATTERN = re.compile(#remompile r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s+" # Tid [2026-10-15 08:30:00] r"\[(?P<register_id>POS_\d+)\]\s+" #Namn på kassan r"item=(?P<item>\w+)\s+" #Namn på ting r"price=(?P<price>\d+\.\d+)\s+" #Pris r"(?P<currency>[A-Z]{3})\s+" #Currency r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)$" #Vilken status har: succuss, refunf eller cancelled) def parse_log(text: str) ->list[dict]: #Function får en input som str #Extract valid transection lines from log text; ignore noise lines records =[] # En list av alla transactioner for line in text.splitlines(): match = LINE_PATTERN.match(line.strip()) #delete alla onödiga karaktärer if match: record = match.groupdict() #Ge tillbake alla grupper som har namn record["price"] = float(record["price"]) records.append(record) return records

Response: Koden ser riktigt bra ut och lösningen med `named groups` (`?P<...>`) och `.groupdict()` är både ren och lättläst.

Här är en genomgång av koden:

- **Styrkor i implementationen:**
  - Du använder `re.compile()` i förväg, vilket sparar prestanda vid många anrop.
  - `(?P<name>...)` matchar exakt de 6 fält som krävs (`timestamp`, `register_id`, `item`, `price`, `currency`, `status`).
  - `price` konverteras korrekt till `float`.
  - Bruslinjer (som saknar formatet) matchar inte och filtreras bort automatiskt.
- **En liten detalj att hålla koll på:**
  - I regex-mönstret har du `(?P<register_id>POS_\d+)`. I `generate_data_for_d2.py` heter kassorna `POS_01`, `POS_02` osv., så där fungerar det klockrent. Om uppgiftsrättningen skulle testa mot en kassa som inte heter just `POS_xx` (t.ex. `REGISTER_1` eller `REG_A`), kan du göra den aningen mer generell med `\[(?P<register_id>[^\]]+)\]`. Men för den givna datan fungerar din variant utan problem.

Du kan snabbt testa koden på texten du läste in tidigare:

```
parsed_records = parse_log(text)
print(f"Hittade {len(parsed_records)} transaktioner.")
print("Exempel:", parsed_records[0] if parsed_records else "Inga rader hittades")
```

Nu är **Task 1** helt klar!

Nästa steg är **Task 2: Recursive utility functions.** Där ska vi skriva två rekursiva funktioner:

1. `find_log_files`: Söka rekursivt efter alla `.log`-filer med `Path.iterdir()` (inga `glob`, `rglob` eller `os.walk`).
2. `get_active_registers`: Traversera `data/store_config.json` rekursivt och returnera ett `set[str]` med alla register-ID:n där `"active": true`.

Vill du gå vidare och skriva den första rekursiva funktionen (`find_log_files`) för Task 2.1 nu?

User prompt: #Task 2.1 def find_log_files(directory) -> list[Path]: # Recursively find all .log files in a directory and its subdirectories directory = Path(directory) # Det kasnke vara en str som data då ändrar vi found = [] for item in directory.iterdir(): if item.is_dir(): found.extend(find_log_files(item)) elif item.is_file() and item.suffix == ".log": found.append(item) return found

Response: Den koden är klockren!

Här stämmer alla detaljer mot specifikationen för Task 2.1:

- Du konverterar input till `pathlib.Path`.
- Du itererar med `iterdir()`.
- Du anropar funktionen rekursivt med `found.extend(find_log_files(item))` när objektet är en mapp.
- Du filtrerar på filer med `.log`.
- Du respekterar begränsningen att varken använda `glob()`, `rglob()` eller `os.walk()`.

Du kan snabbt verifiera den i notebooken med:

```
log_files = find_log_files("data/2026")
print(f"Hittade {len(log_files)} loggfiler.")
for p in log_files[:3]:
    print(p)
```

Eftersom scriptet genererar 3 butiker med 3 kassaapparater vardera bör du få fram exakt 9 loggfiler.

Nu är det dags för **Task 2.2: Rekursiv extrahering från `store_config.json`**. Kraven för funktionen:

- Den ska kunna hantera djupt nästlade datastrukturer (en blandning av `dict` och `list`).
- Returnera ett `set[str]` med alla register-id:n där `"active": True`.

Vill du skriva funktionen för Task 2.2 nu och skicka över den?

User prompt: log_files = find_log_files("data") print("Number of log files:", len(log_files)) for f in log_files: print(f)

Response: Att köra den direkt på `data` är ett perfekt test! Den hittar alla underkataloger genom rekursionen och skriver ut alla `.log`-filer. Du bör få 9 filer utskrivna (3 butiker × 3 kassor).

Nu kan du ta tag i **Task 2.2: Rekursiv extrahering från `store_config.json`**.

Krav för Task 2.2:

- Skriv en funktion som traverserar godtyckliga kombinationer av nästlade `dict` och `list`.
- Identifiera objekt som representerar ett register där `"active": True` (i exemplet har de nycklarna `"id"` och `"active"`).
- Returnera ett `set[str]` med ID-strängarna för alla aktiva register (type hint: `set[str]`).

Strukturen brukar se ut ungefär så här i pseudokod:

```
def get_active_registers(data) -> set[str]:
    active_ids = set()
    # Om data är en dict:
```

```
    # 1. Kolla om data har 'id' och 'active' == True -> lägg till i active_ids
    # 2. Rekursera över alla värden i dict:en (data.values())
    # Om data är en list:
    # 3. Rekursera över varje element i listan
    return active_ids
```

Vill du skriva klart implementationen för Task 2.2 och klistra in den här?

User prompt: #Task 2.2 def find_active_registers(data) -> set[str]: # Recursively collect ids of all registers with "active": true. active =set() if isinstance(data, dict): if "id" in data and data.get("active") is True: active.add(data["id"]) for value in data.values(): active |= find_active_registers(value) elif isinstance(data, list): for element in data: active |= find_active_registers(element) return active

Response: Koden är helt rätt och elegant skriven.

Flera detaljer är särskilt väl genomtänkta:

- Du hanterar både `dict` och `list` korrekt för godtyckligt nästlade strukturer.
- `active |= find_active_registers(...)` sammanfogar delresultaten till ett `set` med union-operatorn på ett snyggt och pythoniskt sätt.
- `data.get("active") is True` är säkert och undviker krascher om nyckeln saknas.
- Typannoteringen `set[str]` matchar uppgiftsspecifikationen exakt.

Du kan testa den mot `store_config.json` med följande rader:

```
with open("data/store_config.json", "r", encoding="utf-8") as f:
    config_data = json.load(f)

active_regs = find_active_registers(config_data)
print("Aktiva register:", active_regs)
```

I `generate_data_for_d2.py` finns `POS_01`, `POS_02` och `POS_03` satta till `true` och `POS_04` satt till `false`, så du bör få ut `{'POS_01', 'POS_02', 'POS_03'}`.

Nu är **Task 2** helt klar.

Nästa steg är **Task 3: Functions for cleaning and analysing data using NumPy**:

1. **Task 3.1 (`clean_price_outliers` / `remove_price_outliers`):**
    - Konvertera indata till en 1D NumPy float-array.
    - Hitta outliers där avvikelsen från medelvärdet är strikt större än 3 standardavvikelser: $|x - \mu| > 3\sigma$.
    - Beräkna medianen för alla **giltiga** priser (dvs. utan outliers).
    - Ersätt alla identifierade outliers med denna median.
    - Returnera den rengjorda arrayen.
2. **Task 3.2 (`compute_refund_ratio`):**
    - Ta emot `sales_count: np.ndarray` och `refund_count: np.ndarray` och returnera en `np.ndarray`.

- Beräkna formeln elementvis: $\text{Refund Ratio} = \frac{\text{Refund Count}}{\text{Sales Count}+10^{-6}}$.

Vill du börja med outlier-funktionen för Task 3.1 och klistra in ditt utkast här?

User prompt: config_path = Path("data") / "store_config.json" config = json.loads(config_path.read_text(encoding="utf-8")) active_registers = find_active_registers(config) print("Active registers:", active_registers)

Response: Det där är ett mycket smidigt sätt att läsa in JSON-filen med `pathlib` och `json.loads`!

Utskriften ska visa de aktiva kassorna från konfigurationen (dvs. `{'POS_01', 'POS_02', 'POS_03'}`). Därmed är **Task 2** helt klar och verifierad.

Nu går vi vidare till **Task 3: Functions for cleaning and analysing data using NumPy**:

- **Task 3.1: Ta bort pris-outliers**
  - Konvertera prisdatan till en 1D NumPy float-array (`np.asarray(..., dtype=float)`).
  - Beräkna medelvärde (μ) och standardavvikelse (σ) med `np.mean()` och `np.std()`.
  - Skapa en mask för outliers: $|x - \mu| > 3\sigma$ (använd `np.abs()`).
  - Beräkna medianen för de giltiga priserna (där masken är `False`, t.ex. `np.median(prices[~outlier_mask])`).
  - Ersätt värdena där masken är `True` med medianen.
  - Returnera den rengjorda arrayen.

Vill du skriva funktionen för Task 3.1 och klistra in koden här?

---

*User prompt: #Task 3.1 def remove_outliers(prices) -> np.ndarray: #Replace value more than 3 standard deviations from the mean with the median of valid values x = np.asarray(prices, dtype=float).ravel() mean = x.mean() std = x.std() is_outlier = np.abs(x - mean) > 3 * std median_valid = np.median(x[~is_outlier]) return np.where(is_outlier, median_valid, x)*

Response: Koden för Task 3.1 är helt rätt och följer specifikationen i detalj:

- `np.asarray(..., dtype=float).ravel()` säkerställer en 1D float-array.
- `np.abs(x - mean) > 3 * std` implementerar $|x - \mu| > 3\sigma$ korrekt.
- `np.median(x[~is_outlier])` beräknar medianen uteslutande över de giltiga priserna.
- `np.where(is_outlier, median_valid, x)` ersätter outliers utan att mutera ursprungsvariabeln oavsiktligt.

Du kan testa den snabbt med ett litet syntetiskt exempel:

```
test_data = np.array([4.5, 4.2, 5.0, 999.0, 4.8, 3.9])
cleaned = remove_outliers(test_data)
print("Före:", test_data)
print("Efter:", cleaned)
```

Nu kan du ta **Task 3.2: `compute_refund_ratio`**.

Krav för Task 3.2:

- Funktionssignatur: `compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray`.
- Formel:

$$\text{Refund Ratio} = \frac{\text{Refund Count}}{\text{Sales Count} + 10^{-6}}$$

- Beräkningen ska ske elementvis med NumPy-vektorisering.

Vill du skriva funktionen för Task 3.2 och klistra in koden här?

User prompt: #Task 3.2 def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray: # Element-wise refund ratio: epsilon avoids division by zero sales = np.asarray(sales_count, dtype = float) refunds = np.asarray(refund_count, dtype = float) return refunds / (sales + 1e-6)

Response: Koden för Task 3.2 är helt korrekt och implementerar formeln exakt som den specificerats i uppgiften:

- Typannoteringarna matchar instruktionerna.
- Att konvertera till `np.asarray(..., dtype=float)` säkerställer flyttalsdivision även om indata skickas in som heltal.
- `sales + 1e-6` förhindrar division med noll.
- Beräkningen är fullt vektoriserad via elementvisa NumPy-operationer.

Du kan testa den snabbt med:

```
s = np.array([100, 50, 0])
r = np.array([5, 2, 0])
print("Refund ratios:", compute_refund_ratio(s, r))
```

Därmed är **Task 3** helt klar!

Nu går vi vidare till **Task 4: Data analysis using Pandas.** Här ska du skapa fyra funktioner:

1. **Task 4.1 (`create_transaction_df`):**
    - Ta in listan av dictionaries från Task 1.
    - Skapa en `pd.DataFrame`.
    - Konvertera `timestamp` till `datetime` med `pd.to_datetime()` och sätt den som index.
    - Sortera DataFrame kronologiskt efter index.
2. **Task 4.2 (`resample_hourly_revenue`):**
    - Ta emot DataFrame som indata.
    - Gruppera på `item` och kör en tidsbaserad resample på 1 timme (`"1h"`).
    - Summera `price` för att få total intäkt per timme och per artikel.
3. **Task 4.3 (`create_status_pivot`):**

- Skapa en pivot-tabell (`df.pivot_table`) som visar total intäkt (`price`, sum) uppdelad på `item` (rader) och `status` (`SUCCESS`, `REFUND`, `CANCELLED`).

4. **Task 4.4 (`calculate_refund_ratio_per_item`):**
    - Beräkna antalet försäljningar och antalet återköp per vara från DataFrame.
    - Anropa `compute_refund_ratio` från Task 3.
    - Returnera en DataFrame med resultatet per vara.

Vill du börja med Task 4.1 (att bygga och indexera DataFrame) och klistra in din kod här?

User prompt: # Test test_prices = [4.2, 3.1, 7.4, 3.8, 4.5, 3.0] print("Before:", test_prices[-3:]) print("After:", remove_outliers(test_prices)[-3:]) sales = np.array([10, 20, 0]) refunds = np.array([2, 5, 1]) print("Refund ratios:", compute_refund_ratio(sales, refunds))

Response: Testet för `compute_refund_ratio` ser bra ut! Uträkningen för arrayerna ger:

- $2/(10 + 10^{-6}) \approx 0.2$
- $5/(20 + 10^{-6}) \approx 0.25$
- $1/(0 + 10^{-6}) = 10^{6}$

För `remove_outliers` i testet finns det ingen extrem outlier som 999.0 i din lista `test_prices`, så alla värden ligger inom 3 standardavvikelser och förblir oförändrade. Om du vill verifiera att outlier-ersättningen kickar in kan du lägga till ett `999.0`-värde i testlistan:

```
test_with_glitch = [4.2, 3.1, 7.4, 3.8, 4.5, 3.0, 999.0]
print("Cleaned:", remove_outliers(test_with_glitch))
```

Därmed är **Task 3** helt klar!

Nu är vi framme vid **Task 4: Data analysis using Pandas.**

Här är en påminnelse om kraven för deluppgift 4.1:

- Skapa en funktion som tar emot listan med transaktions-dictionaries.
- Konvertera listan till en `pd.DataFrame`.
- Konvertera `timestamp`-kolumnen till `datetime` med `pd.to_datetime()` och sätt den som index.
- Sortera tabellen kronologiskt efter index med `.sort_index()`.
- Returnera din `DataFrame`.

Vill du skriva koden för Task 4.1 och klistra in den här?

User prompt: all_records = [] for f in find_log_files("data"): text, enc = read_log_file(f) all_records.extend(parse_log(text)) print("Total records:", len(all_records))

Response: Den loopen kopplar ihop hela intaget från Task 1 och Task 2.1 på ett mycket smidigt sätt!

Nu har du `all_records` redo, vilket är den perfekta testdatan för **Task 4.1**.

Här är en påminnelse om vad funktionen för **Task 4.1** ska göra:

- Ta emot listan av transaktions-dictionaries (t.ex. `records: list[dict]`).
- Skapa en `pd.DataFrame(records)`.
- Konvertera `timestamp` till datetime (`pd.to_datetime(...)`) och sätt den som index.
- Sortera kronologiskt efter index (`.sort_index()`).
- Returnera din DataFrame.

Vill du skriva funktionen för Task 4.1 och skicka över koden?

User prompt: #Task 4.1 def build_dataframe(records: list[dict]) -> pd.DataFrame: #Convert transaction dicts to a DataFrame indexted and sorted by timestamp df = pd.DataFrame(records) df["timestamp"] = pd.to_datetime(df["timestamp"]) df = df.set_index("timestamp") df = df.sort_index() return df

Response: Koden för Task 4.1 är helt rätt och följer kraven punkt för punkt:

- Konverterar listan av transaktions-dictionaries till en `pd.DataFrame`.
- Konverterar `timestamp` till datetime med `pd.to_datetime()`.
- Sätter `timestamp` som index och sorterar DataFrame kronologiskt via `.sort_index()`.

Du kan snabbt testa den på din `all_records`-variabel:

```
df = build_dataframe(all_records)
print(df.head())
print(df.info())
```

Nu går vi vidare till **Task 4.2: Resampla per timme och artikel**.

Krav för Task 4.2:

- Skriv en funktion som tar emot din DataFrame.
- Gruppera på `item` och resampla över 1-timmarsintervall (`"1h"`) på tidsstämpel-indexet.
- Beräkna summan av `price` för varje timme och artikel.
- *Tips på syntax i moderna Pandas:*

  ```
  # Resample grupperat per kolumn:
  df.groupby("item")["price"].resample("1h").sum()
  ```

  (Notera även att detta senare ska sparas som `hourly_revenue.json` i Task 5).

Vill du skriva funktionen för Task 4.2 och klistra in ditt utkast här?

User prompt: # Task 4.2 def hourly_revenue(df: pd.DataFrame) -> pd.DataFrame: # Total price per item in 1-hour windows (rows = hours, columns = items) hourly = df.groupby("item") ["price"].resample("1h").sum() return hourly.unstack(level="item").fillna(0)

Response: Koden för Task 4.2 är mycket välskriven!

Några styrkor i implementationen:

- `df.groupby("item")["price"].resample("1h").sum()` grupperar och aggregerar korrekt över 1-timmarsintervall per artikel.
- `.unstack(level="item")` skapar en ren matris med timmar som rader och artiklar som kolumner.
- `.fillna(0)` hanterar timmar där en viss artikel inte såldes alls så att du slipper `NaN`-värden.

Testa gärna funktionen med:

```
df_hourly = hourly_revenue(df)
print(df_hourly.head())
```

Nu kan vi ta **Task 4.3: Pivot-tabell över status**.

Krav för Task 4.3:

- Skapa en funktion som tar emot din DataFrame.
- Skapa en pivot-tabell (`df.pivot_table`) som visar den totala transaktionsintäkten (`price`, sum) uppdelad på `item` och `status` (`SUCCESS`, `REFUND`, `CANCELLED`).
- `item` ska utgöra raderna och `status` kolumnerna (använd gärna `fill_value=0` för eventuella saknade kombinationer).

Vill du skriva funktionen för Task 4.3 och klistra in koden här?

User prompt: # Task 4.3 def status_pivot(df:pd.DataFrame) -> pd.DataFrame: #Total revenue per item (rows) for each statue(columns) return pd.pivot_table(df, values = "price", index = "item", columns = "status", aggfunc = "sum", fill_value = 0,)

Response: Koden för Task 4.3 är helt korrekt implementerad!

- `values="price"` och `aggfunc="sum"` summerar den totala transaktionsintäkten.
- `index="item"` och `columns="status"` grupperar artiklar som rader mot statuskategorierna som kolumner.
- `fill_value=0` garanterar att eventuella tomma celler fylls med noll istället för `NaN`.

Testa den gärna med:

```
pivot = status_pivot(df)
print(pivot)
```

Nu tar vi den sista funktionen i Fas 4: **Task 4.4: Refund ratio per artikel**.

Krav för Task 4.4:

- Ta emot din DataFrame.
- Beräkna antalet försäljningar (där `status == "SUCCESS"`) och antalet återköp (där `status == "REFUND"`) per artikel.
- Skicka dessa antal som NumPy-arrays till din funktion `compute_refund_ratio` från Task 3.
- Returnera en `pd.DataFrame` eller `pd.Series` (en DataFrame med artiklar och kolumnen `refund_ratio` är perfekt för att spara till `item_refund_ratios.csv` senare).

Vill du skriva funktionen för Task 4.4 och skicka över koden?

User prompt: #Task 4.4 def item_refund_ratios(df:pd.DataFrame) -> pd.DataFrame: #refund ratio per item, using compute_refund_ratio from task 3. counts = pd.crosstab(df["item"], df["status"]) counts = counts.reindex(columns=["SUCCESS", "REFUND", "CANCELLED"], fill_value=0) ratio = compute_refund_ratio(counts["SUCCESS"].to_numpy(), counts["REFUND"].to_numpy(),) return pd.DataFrame({ "sales_count": counts["SUCCESS"], "refund_count": counts["REFUND"], "refund_ratio": ratio, })

Response: Koden för Task 4.4 är mycket väl genomtänkt och strukturerad!

Flera detaljer är särskilt starka:

- `pd.crosstab(df["item"], df["status"])` räknar effektivt förekomsten av varje status per vara.
- `.reindex(columns=["SUCCESS", "REFUND", "CANCELLED"], fill_value=0)` förhindrar fel om en viss statuskategori saknas helt i datan.
- `.to_numpy()` skickar arrayer direkt till `compute_refund_ratio` i linje med instruktionerna.
- Resultat-DataFramen innehåller både basantalen och den beräknade kvoten.

Testa gärna funktionen med:

```
ratios = item_refund_ratios(df)
print(ratios)
```

Därmed är **Task 4** helt färdig!

Nu återstår **Task 5: Create full workflow** där du ska knyta samman alla delar och generera output-filerna.

Arbetsflödet ska utföra följande steg i ordning:

1. Hitta alla `.log`-filer rekursivt och hämta aktiva register-ID:n från `store_config.json`.
2. Läs in och parsa loggarna till en lista av dictionaries.
3. Bygg den initiala `DataFrame`:n.
4. Rengör pris-outliers ($999.00) med NumPy-funktionen `remove_outliers`.
5. Korrigera valuta: multiplicera `price` med 10.0 där `currency == "NOK"`.
6. Generera rapporter med dina funktioner:
    - Timvis intäkt (`hourly_revenue`).
    - Pivot-tabell över status (`status_pivot`).
    - Återköpskvot per artikel (`item_refund_ratios`).
7. Spara de 4 filerna i arbetskatalogen:
    - `cleaned_transactions.csv`
    - `hourly_revenue.json`
    - `status_pivot_table.csv`
    - `item_refund_ratios.csv`

Vill du skriva Task 5-skriptet/arbetsflödet och klistra in koden här så att vi kan kontrollera hela pipelinen och fil-exporterna tillsammans?

User prompt: # Test df = build_dataframe(all_records) print(df.head()) print() hourly = hourly_revenue(df) print(hourly) print() pivot= status_pivot(df) print(pivot) print() ratios = item_refund_ratios(df) print(ratios)

Response: Testutskriften kör alla dina analysfunktioner i en följd och verifierar att funktionerna fungerar ihop.

När du nu ska bygga **Task 5: Create full workflow**, är stegen i uppgiften specificerade i en tydlig ordning:

1. **Hitta loggfiler och aktiva register:** Använd `find_log_files("data")` och `find_active_registers(config)`.
2. **Läs och parsa:** Läs in texten med `read_log_file` och samla alla dictionaries från `parse_log`. *(Om uppgiften kräver filtrering på aktiva register, kan du här eller i DataFrame filtrera med `df["register_id"].isin(active_registers)`)*.
3. **Bygg initial DataFrame:** `df = build_dataframe(all_records)`.
4. **Outlier-städning:** Skicka `df["price"]` till `remove_outliers(df["price"].to_numpy())` och skriv tillbaka resultatet till `df["price"]`.
5. **Valutakorrigering (NOK):** Multiplicera priset med 10.0 där `currency == "NOK"`, till exempel:

   ```
   df.loc[df["currency"] == "NOK", "price"] *= 10.0
   ```[cite: 1, 5]
6. **Generera rapporter:** Anropa `hourly_revenue(df)`, `status_pivot(df)` och `item_refund_ratios(df)` på den tvättade och korrigerade DataFramen.
7. **Spara alla 4 filer:**
   - `df.to_csv("cleaned_transactions.csv")`
   - `hourly.to_json("hourly_revenue.json", indent=2, date_format="iso")`
   - `pivot.to_csv("status_pivot_table.csv")`
   - `ratios.to_csv("item_refund_ratios.csv")`

Vill du sätta ihop hela koden för Task 5 i en funktion/cell och klistra in den här så dubbelkollar vi exportformaten och stegen tillsammans?

---

*User prompt: def run_pipeline(data_dir="data", out_dir="output"): #Full workflow: find logs, parse, clean, analyse and save all report data_dir = Path(data_dir) out_dir = Path(out_dir) out_dir.mkdir(exit_ok = True) #1) Find log and active registers log_files = find_log_files(data_dir) config = json.loads((data_dir / "store_config.json").read_text(encoding="utf-8")) active = find_active_registers(config) print(f"Found {len(log_files)} log files. Active registers: {sorted(active)}") #2) read and parse all logs records = [] for f in log_files: text, enc = read_log_file(f) file_records = parse_log(text) records.extend(file_records) print(f"{f} [{enc}] -> {len(file_records)} records") #3) Build DataFrame, keep only avtive registers df = build_dataframe(records) df = df[df["register_id"].isin(active)].copy() #4) remove price outliers (999.00 glitches) n_outliers = (df["price"] == 999.0).sum() df["price"] = remove_outliers(df["price"]) print(f"Replaced {n_outliers} outlier prices") #5) Currency correction for NOK df.loc[df["currency"] == "NOK", "price"] *=10.0 #6) Reports hourly = hourly_revenue(df) pivot = status = status_pivot(df) ratios = item_refund_ratios(df) #7) save output files df.to_csv(out_dir / "cleaned_transactions.csv") hourly_json = hourly.copy() hourly_json.index = hourly_json.index.strftime("%Y-%m-%d %H:%M") hourly_json.to_json(out_dir / "hourly_revenue.json", orient = "index", indent = 2) pivot.to_csv(out_dir /*

"status_pivot_table.csv") ratios.to_csv(out_dir / "item_refund_ratios.csv") print(f"All output files saved to '{out_dir}/'") return df, hourly, pivot, ratios

Response: Koden i `run_pipeline` knyter ihop alla delar och uppfyller stegen från uppgiftsspecifikationen.

Det finns dock två små buggar som gör att koden kommer att krascha vid körning:

Saker att korrigera:

1. **Stavfel i `mkdir`:**
    - På rad 5 har du skrivit `exit_ok = True`. Parametern i `pathlib.Path.mkdir` heter `exist_ok=True`.
2. **Dubbel tilldelning:**
    - På rad 31 har du skrivit `pivot = status = status_pivot(df)`. Även om det fungerar i Python är variabeln `status` onödig och kan orsaka förvirring. Det räcker med `pivot = status_pivot(df)`.
3. **Placering av output-filer:**
    - Enligt specifikationen ska filerna genereras i projektets arbetskatalog, inte nödvändigtvis i en separat undermapp `output/`. Sätter du `out_dir="."` sparas de direkt i arbetskatalogen enligt instruktionerna.

Korrigerad version:

```
def run_pipeline(data_dir="data", out_dir="."):
    # Full workflow: find logs, parse, clean, analyse and save all reports
    data_dir = Path(data_dir)
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    # 1) Find logs and active registers
    log_files = find_log_files(data_dir)
    config = json.loads((data_dir / "store_config.json").read_text(encoding="utf-8"))
    active = find_active_registers(config)
    print(f"Found {len(log_files)} log files. Active registers: {sorted(active)}")

    # 2) Read and parse all logs
    records = []
    for f in log_files:
        text, enc = read_log_file(f)
        file_records = parse_log(text)
        records.extend(file_records)
        print(f"{f} [{enc}] -> {len(file_records)} records")

    # 3) Build DataFrame, keep only active registers
    df = build_dataframe(records)
    df = df[df["register_id"].isin(active)].copy()

    # 4) Remove price outliers (999.00 glitches)
    n_outliers = (df["price"] == 999.0).sum()
    df["price"] = remove_outliers(df["price"])
    print(f"Replaced {n_outliers} outlier prices")

    # 5) Currency correction for NOK
    df.loc[df["currency"] == "NOK", "price"] *= 10.0
```

```
    # 6) Reports
    hourly = hourly_revenue(df)
    pivot = status_pivot(df)
    ratios = item_refund_ratios(df)

    # 7) Save output files
    df.to_csv(out_dir / "cleaned_transactions.csv")

    hourly_json = hourly.copy()
    hourly_json.index = hourly_json.index.strftime("%Y-%m-%d %H:%M:%S")
    hourly_json.to_json(out_dir / "hourly_revenue.json", orient="index", indent=2)

    pivot.to_csv(out_dir / "status_pivot_table.csv")
    ratios.to_csv(out_dir / "item_refund_ratios.csv")
    print(f"All output files saved to '{out_dir}/'")

    return df, hourly, pivot, ratios
```

Kör nu funktionen i en notebook-cell:

```
df, hourly, pivot, ratios = run_pipeline()
```

Kontrollera att de 4 filerna (`cleaned_transactions.csv`, `hourly_revenue.json`, `status_pivot_table.csv`, `item_refund_ratios.csv`) har skapats och ser korrekta ut i din filutforskare.

Skapades alla fyra filer utan fel när du körde `run_pipeline()`, och vill du gå igenom inlämningskraven (git-logg, PDF-export och GenAI-redogörelse)?

User prompt: Jag har gjort git-logg og genai delen själv

Response: Vad bra att du redan har ordnat git-loggen och GenAI-redogörelsen! Då har du bockat av två viktiga delar i inlämningskraven.

Här är en sista kontrollista för att säkerställa att allt är komplett inför inlämningen:

- **Jupyter Notebook (`.ipynb`) och PDF:**
    - Kör om hela notebooken från början till slut (`Kernel -> Restart & Run All`) så att alla celler körs i ordning och att inga fel uppstår.
    - Säkerställ att diskussionsdelar, förklaringar och eventuella grafer/utskrifter syns tydligt i markdown- och kodcellerna.
    - Exportera notebooken till en `.pdf`-fil.
- **Genererade output-filer:**
    - Kontrollera att alla 4 filer finns med och är skapade:
        - `cleaned_transactions.csv`
        - `hourly_revenue.json`
        - `status_pivot_table.csv`
        - `item_refund_ratios.csv`

- **GitHub Repository:**
  - Länk till ert GitHub-repo i notebooken/inlämningen.
  - Filen `git_log.txt` genererad via kommandot från instruktionen:

    ```
    git log --graph --pretty=format:"%h - %an, %ar : %s" > git_log.txt
    ```

  - En `requirements.txt` som speglar den virtuella miljön.
- **Kort video (ca 5 minuter):**
  - Spela in presentationsvideon där båda i gruppen medverkar och förklarar koden och resultaten.

Behöver du hjälp med att se över formatet på någon av de exporterade filerna, eller känner du dig redo att paketera och lämna in?

User prompt: Vad ska jag göra med pdf-filen

Response: Enligt instruktionerna i uppgiften under **Hand-in information** ska du göra följande med PDF-filen:

1. **Exportera din färdiga Jupyter Notebook till PDF:**
   - När all kod är körd och alla resultat och markdown-texter syns, exporterar du hela notebooken till en `.pdf`-fil.
   - I Jupyter Notebook / JupyterLab gör du detta enklast via:

     ```
     File -> Save and Export Notebook As... -> PDF
     ```

     *(Om du inte har LaTeX/pandoc installerat lokalt kan du välja `File -> Print...` i webbläsaren och välja **"Spara som PDF" / "Save as PDF"** – se till att bakgrundsgrafik är ikryssat så att kodrutorna syns tydligt).*
2. **Lämna in PDF-filen tillsammans med `.ipynb`-filen:**
   - Du ska ladda upp **både** `.ipynb`-filen och PDF-kopian av notebooken i er inlämning (på er kursplattform, t.ex. Canvas eller Blackboard).
   - PDF-filen fungerar som en statisk version av inlämningen så att lärarna och examinatorerna enkelt kan läsa och rätta era resultat, koder och diskussioner utan att behöva starta upp en Jupyter-server lokalt.

Har du problem med att exportera notebooken till PDF direkt från Jupyter, eller vill du ha tips på hur du sparar den snyggt via webbläsaren?

User prompt: Hur kan jag fixar en pdf fil från vscode?

Response: Det finns tre enkla sätt att skapa en PDF från en `.ipynb`-notebook i VS Code, beroende på vilka verktyg du har installerade.

Metod 1: Via "Export" i verktygsfältet (Snabbast)

1. Öppna din `.ipynb`-fil i VS Code.

2. I menyraden precis ovanför notebook-cellerna (där du ser knappar som *Run All*, *Restart* etc.), klicka på knappen med tre punkter (`...`) längst till höger.
3. Välj **Export** och klicka på **PDF**.
4. Om VS Code saknar `nbconvert` eller `pandoc` får du en fråga om att installera dem. Om det inte fungerar direkt eller kräver LaTeX, använd **Metod 2**.

Metod 2: Exportera till HTML och spara som PDF (Mest pålitligt)

Detta är den enklaste metoden eftersom den inte kräver någon LaTeX-installation och ger en layout som ser exakt ut som i VS Code:

1. Klicka på de tre punkterna (`...`) längst upp i notebooken i VS Code.
2. Välj **Export** → **HTML**.
3. Spara HTML-filen och öppna den i din vanliga webbläsare (Chrome, Edge eller Safari).
4. Tryck `Ctrl + P` (Windows) eller `Cmd + P` (Mac) för att skriva ut.
5. Välj **Spara som PDF** som skrivare/destination.
6. Kryssa i **Bakgrundsgrafik** under fler inställningar så att färger och kodblock syns korrekt, och klicka på **Spara**.

Metod 3: Via terminalen med `nbconvert`

Om du har `nbconvert` installerat i din virtuella miljö kan du köra ett terminalkommando direkt i VS Code:

För att generera HTML och sedan skriva ut till PDF:

```
jupyter nbconvert --to html deliverable_2.ipynb
```

Öppna den genererade HTML-filen och skriv ut till PDF via webbläsaren.

Om du har `pyppeteer` installerat kan du konvertera direkt till PDF via webbläsarmotorn utan LaTeX:

```
jupyter nbconvert --to webpdf deliverable_2.ipynb
```

Fick du något felmeddelande när du provade att exportera via de tre punkterna i VS Code, eller vill du använda webbläsarmetoden via HTML?
